COMPSYS 704 Individual Project - Milestone 1 Interim Report

Hierarchical Read-Only Process Visualisation with Controller-Level Drill-Down and a Shared Observer-Side Process Model

Repository-Aligned Implementation Audit, Current Contribution and Future Read-Only Extensions

1. Introduction and Motivation

The COMPSYS 704 Automated Bottling System (ABS) is a distributed reactive production-line simulation in which loading, transport, rotary positioning, filling, lid placement, capping and unloading may progress concurrently. SystemJ supports this kind of multiclock GALS organisation, in which locally synchronous Clock Domains interact asynchronously [1]. The 2026 Group Project already requires simple visualisation of machine operation, machine status and workpiece/progress representation. A basic status panel or whole-line display is therefore the Group Project observation baseline, not the main Individual Project contribution. This IP extends that baseline into a hierarchical, read-only process visualisation with a Level-1 production overview, Level-2 controller/module drill-down, continuous idealised mechanism animations and one shared observer-side process model that keeps all views consistent. This overview-to-detail structure is consistent with established industrial HMI practice, where higher-level displays preserve process context and drill-down views expose focused equipment detail [2], [3].

The repository shows a clear progression from ten Coordinator-provided VIZ_* values, through a spatial ProductionLinePanel, to the current hierarchical GUI. Machine regions are clickable; each implemented module opens or reuses a dedicated JDialog detail view; and VisualisationSyncModel shares visual-only bottle/process state with the overview and the per-module DetailAnimationModel objects. POS order identifiers advance automatically after completion to simplify repeated demonstrations, but that behaviour is only an integration convenience and is not presented as the core visualisation contribution.

2. Proposed Individual Project

The IP is organised around three linked elements. Level 1 is the integrated whole-line overview showing the implemented ABS modules, their latest confirmed status and batch progress. Level 2 is the controller/module detail layer opened by clicking a machine region. Under both levels, VisualisationSyncModel creates observer-side VirtualBottle records from the required batch count and shares their idealised process state with ProductionLinePanel and the relevant DetailAnimationModel. This prevents the overview and detail dialogs from independently simulating contradictory bottles or mechanism phases.

The implemented overview contains eight monitored units: Bottle Loader, Conveyor, Rotary Table, Filler A, Filler B, Lid Loader, Capper and Bottle Unloader. The two Conveyor drawings represent different portions of one monitored Conveyor and share VIZ_CONVEYOR_STATUS. The group architecture now places P6 Labelling after Capping and before Unloading; however, no approved Coordinator-facing LABELLER_STATUS_REQUEST / LABELLER_STATUS boundary or VIZ_LABELLER_STATUS exists in the current M1 source/XML, so the Labeller is not shown as a real monitored module. Downstream M4 Sort/Pack may be visualised later, but it is not part of the current implementation and does not determine POS completion.

3. Problem and Technical Challenges

The first challenge is preserving a truthful boundary between distributed-system observation and smooth visual explanation. Controller Clock Domains update independently while Coordinator polling, status reception, batch counting and VIZ forwarding run concurrently. This concurrency is consistent with the GALS execution model supported by SystemJ [1]. The GUI consumes the latest confirmed values without participating in sequencing or actuation. It distinguishes the initial display-only WAITING condition from the shared status codes 0 = IDLE, 1 = READY, 2 = BUSY, 3 = DONE and 4 = FAULT; WAITING only means that no valid status has yet been observed.

The second challenge is consistency under limited telemetry. The current accepted M1-facing observation boundary provides real confirmed machine status, required bottle count and completed bottle count, but not exact bottle position, rotary angle, conveyor displacement, valve state, measured fill percentage, lid/capper displacement, physical phase or bottleId/workpieceId. The implementation therefore separates REAL OBSERVATION STATE from an IDEALISED OBSERVER-SIDE PROCESS STATE. VisualisationSyncModel uses shared visual-only records, constrains idealised progress when FAULT is confirmed and treats VIZ_COMPLETED_BOTTLES as a real completion anchor. These values explain the process visually; they are not a Digital Twin and are not claimed as physical truth. This conservative terminology is consistent with manufacturing literature that distinguishes one-way digital representations from more tightly coupled Digital Twin concepts [4], [5].

4. Conceptual Design

The architecture keeps control and observation separate. Machine Controllers and Plants retain local sequencing, safety and actuator authority. CoordinatorCD polls and stores their latest status, tracks required/completed bottles, and forwards ten Integer VIZ_* values one way to ABSVisualisationPlantCD:11008. ABSVisualisation caches those confirmed values, while VisualisationSyncModel and the module-specific DetailAnimationModel objects create the idealised read-only process state used by ProductionLinePanel and reusable JDialog detail views. Neither the cache nor either visual model sends commands, state changes or actuator requests back to CoordinatorCD or any Controller.

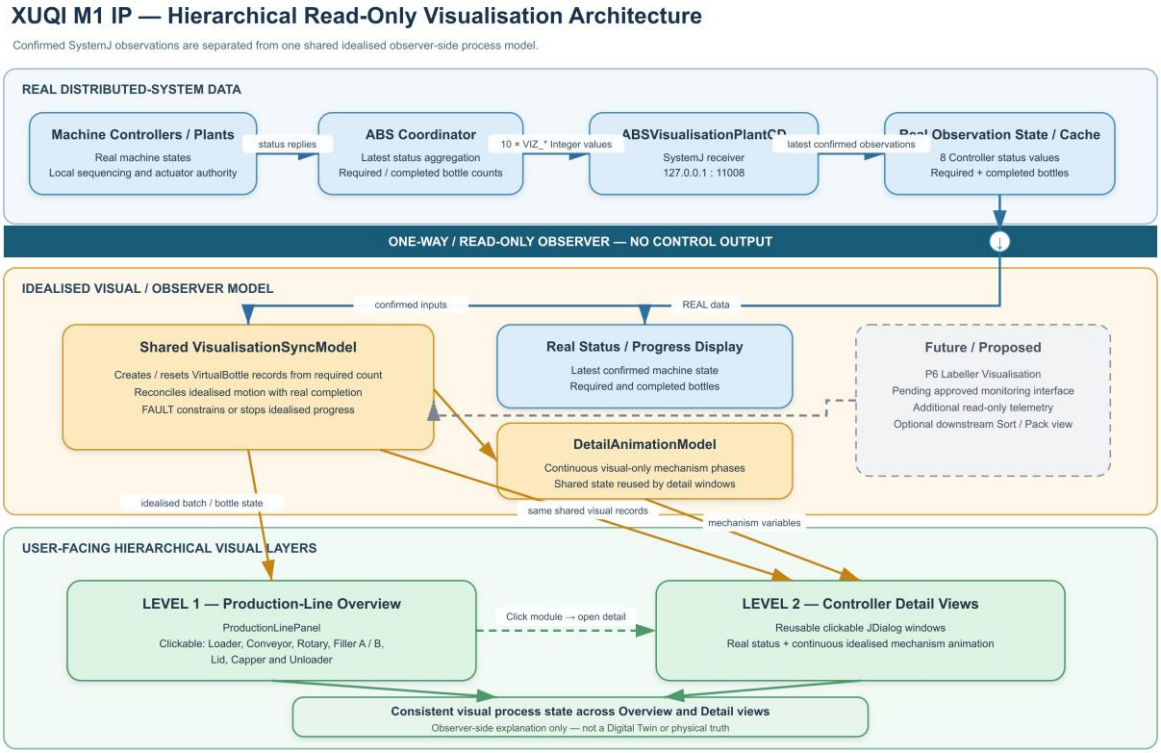

Figure 1. Hierarchical read-only visualisation architecture. Confirmed Coordinator-provided observation data is separated from the shared observer-side process model that drives the Level-1 overview and Level-2 controller detail views.

4.1 System Interfaces

The Coordinator-facing table below records the current accepted M1-facing integration boundary, including later cross-member updates rather than only the original Integration V1 wording. Status requests are supervisory and must not start, acknowledge, reset or advance physical work. All current status replies and BOTTLE_DONE return to CoordinatorCD:11001. The IP consumes Coordinator-forwarded observations only and does not change these machine-control contracts; future detail must use separately agreed read-only telemetry.

Machine	Coordinator -> Controller	Controller -> Coordinator	Ports*
Bottle Loader	`START_ORDER (Integer); LOADER_STATUS_REQUEST (pure)`	`LOADER_STATUS (Integer)`	11002 / 11001
Conveyor	`CONVEYOR_STATUS_REQUEST (pure)`	`CONVEYOR_STATUS (Integer)`	11009 / 11001
Rotary Table	`ROTARY_STATUS_REQUEST (pure)`	`ROTARY_STATUS (Integer)`	11003 / 11001
Filler A	`FILL_A_RATIO (Integer); FILLER_A_STATUS_REQUEST (pure)`	`FILLER_A_STATUS (Integer)`	11004 / 11001
Filler B	`FILL_B_RATIO (Integer); FILLER_B_STATUS_REQUEST (pure)`	`FILLER_B_STATUS (Integer)`	11005 / 11001
Lid Loader	`LID_STATUS_REQUEST (pure)`	`LID_STATUS (Integer)`	11006 / 11001
Capper	`CAPPER_STATUS_REQUEST (pure)`	`CAPPER_STATUS (Integer)`	11007 / 11001
Bottle Unloader	`UNLOADER_STATUS_REQUEST (pure)`	`UNLOADER_STATUS (Integer); BOTTLE_DONE (pure)`	11010 / 11001
Coordinator receiver	–	`All current *_STATUS replies; BOTTLE_DONE`	— / 11001

** Ports are shown as machine receiver / Coordinator receiver. Labeller monitoring remains proposed and is intentionally excluded from this implemented table.*

4.2 Visualisation Interface

The implemented visualisation boundary is exactly ten Integer outputs from CoordinatorCD to ABSVisualisationPlantCD at 127.0.0.1:11008: eight latest machine-status values plus VIZ_REQUIRED_BOTTLES and VIZ_COMPLETED_BOTTLES. VIZ_REQUIRED_BOTTLES initialises or

resets the shared visual batch; VIZ_COMPLETED_BOTTLES anchors final observer-side completion so the idealised Unloader cannot finalise more bottles than the real system has reported complete. The interface carries no recipe percentages, bottle identities, measured positions, mechanism phases or actuator state.

Visualisation signal	Type	Meaning
VIZ_LOADER_STATUS	Integer	Loader state
VIZ_CONVEYOR_STATUS	Integer	Conveyor state
VIZ_ROTARY_STATUS	Integer	Rotary Table state
VIZ_FILLER_A_STATUS	Integer	Filler A state
VIZ_FILLER_B_STATUS	Integer	Filler B state
VIZ_LID_STATUS	Integer	Lid Loader state
VIZ_CAPPER_STATUS	Integer	Capper state
VIZ_UNLOADER_STATUS	Integer	Bottle Unloader state
VIZ_REQUIRED_BOTTLES	Integer	Required bottles in current batch
VIZ_COMPLETED_BOTTLES	Integer	Completed bottles in current batch

The overview draws input and output Conveyor portions, but both use the same VIZ_CONVEYOR_STATUS and do not imply a second Conveyor Controller. Likewise, displayed bottle numbers are shared observer-side identifiers used to reconcile Overview and Detail views. They are not real bottleId/workpieceId telemetry, do not prove exact position and never enter SystemJ control.

4.3 Planned Interface Evolution

Status-only data limits the accuracy of detailed animation. Future team-agreed read-only observation extensions may include active recipe percentages, machine phase/sub-state, Rotary position/angle, bottle-present transfer observations, measured or simulated fill levels, Lid/Capper sub-state, bottle/workpiece identifier, P6 Labeller status/information and downstream Sort/Pack status. These are proposed observation extensions, not unilateral changes to control interfaces. P6 Labelling is already placed after Capping and before Unloading, but its Coordinator-facing monitoring signals and VIZ_LABELLER_STATUS remain proposed and unimplemented.

5. Development History and Current Implementation

Development has four practical stages. First, Coordinator polling and ten VIZ_* values established the read-only status/progress path. Second, a Graphics2D ProductionLinePanel replaced the vertical status list with a spatial whole-line view. Third, eight machine regions became clickable and reusable JDialog detail windows were added for Loader, Conveyor, Rotary, Filler A, Filler B, Lid, Capper and Unloader. Their gate, belt, rotary, fill, lid, capper-head and unload graphics progress in small continuous idealised increments instead of large discrete jumps.

The fourth stage adds one reconciliation model instead of letting each window invent its own bottle. VIZ_REQUIRED_BOTTLES creates a batch-sized set of VirtualBottle records; the same observer-side identity is reused through the overview and DetailAnimationModel views; and real completion can trigger bounded visual catch-up. Unloader visual completion waits for VIZ_COMPLETED_BOTTLES, while any confirmed FAULT stops or constrains idealised movement. The current code still uses a five-station, 72-degree observer-side Rotary model and the overview labels it '5 symbolic stations'. This is a simplified pre-P6 visual representation, not the real six-position group architecture, and alignment with six positions/P6 remains a refinement rather than evidence of real bottle tracking.

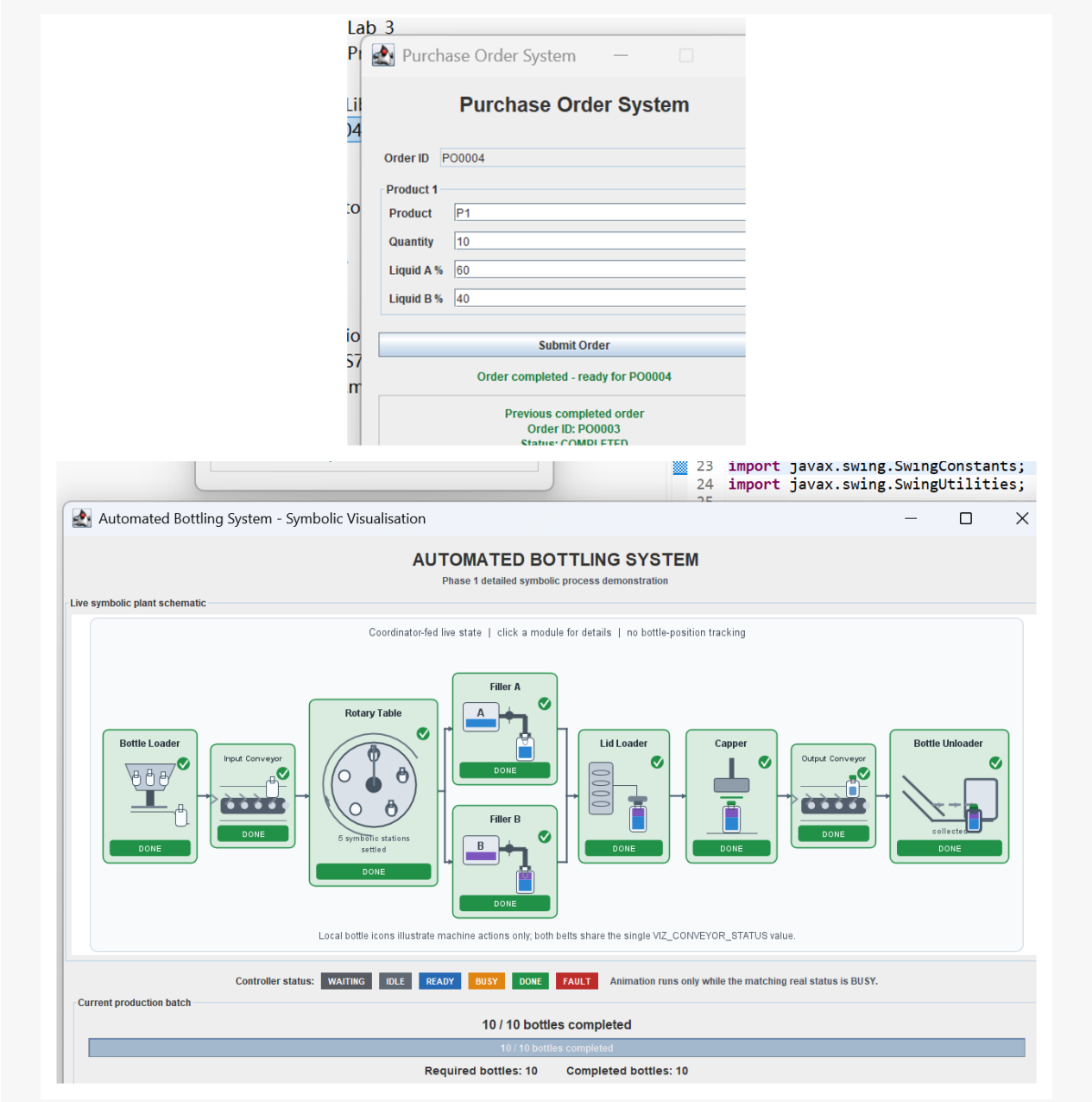

Figure 2. Current Level-1 integrated overview. Confirmed Controller status and batch counts are shown with shared idealised bottle-flow state; icons and positions are observer-side representations, not tracked workpieces.

6. Planned Development: Telemetry Refinement and P6 Architecture Alignment

The next stage is to replace selected idealised variables with approved real Controller/Plant observations and align the current five-station symbolic Rotary representation with the accepted six-position architecture. P6 Labelling is already located after Capping and before Unloading, but M1 must wait for a frozen and verified Coordinator-facing Labeller monitoring boundary before adding a real Labeller module or VIZ_LABELLER_STATUS. After approval, a Level-2 Labeller view may expose confirmed label status/information without gaining control authority. A downstream Sort/Pack view is also optional future work and must not take ownership of BOTTLE_DONE or POS order completion.

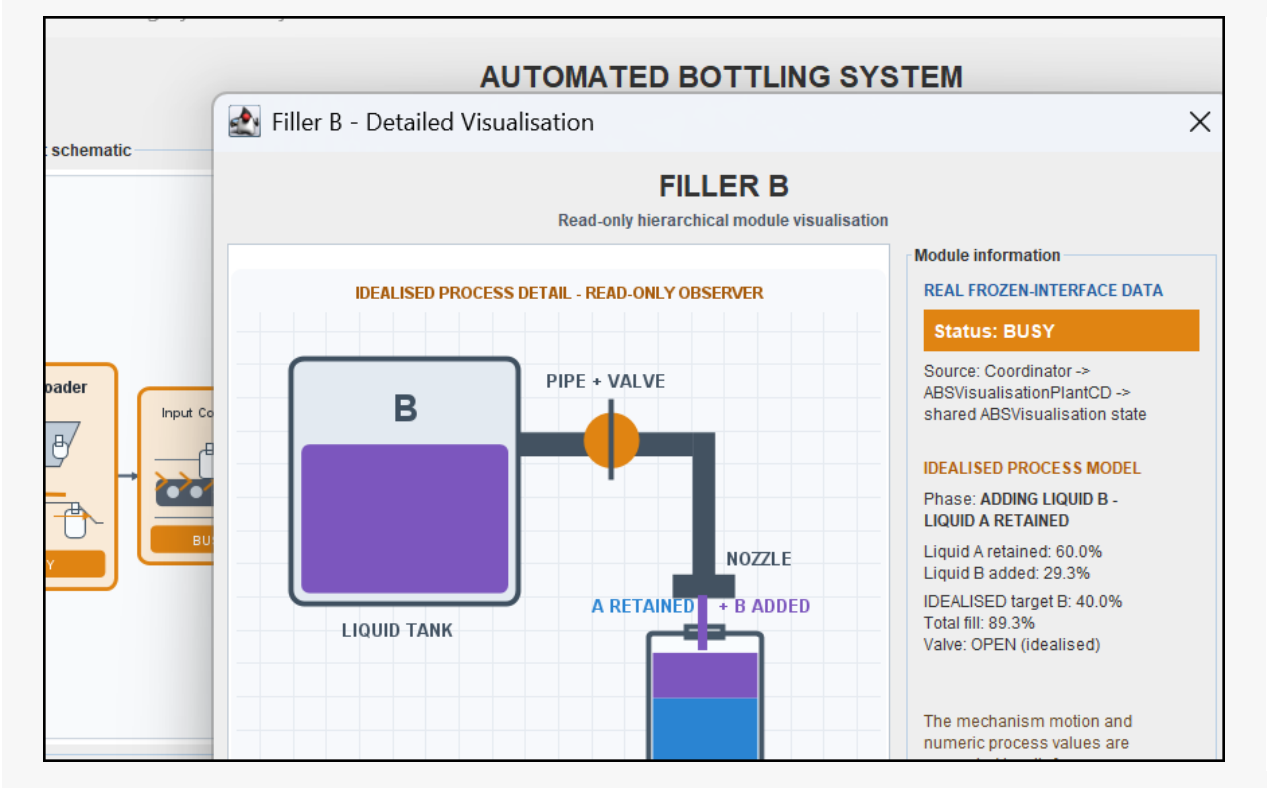

Figure 3. Current Level-2 Filler B detail view. The real BUSY/DONE status is separated explicitly from the idealised liquid-level, valve and shared visual-bottle information.

7. Expected Contribution and Future Refinement

The expected contribution comprises: (1) hierarchical navigation from a Level-1 plant view to Level-2 machine detail; (2) continuous controller/module mechanism visualisation; (3) one reconciled observer-side process model shared across all windows; (4) explicit provenance separating latest confirmed SystemJ observations from idealised visual variables; (5) non-invasive, one-way integration with no control output; and (6) an extensible structure for future team-approved read-only telemetry. The Level-1-to-Level-2 navigation is motivated by established HMI display-hierarchy principles rather than a claim of formal ISA-101 compliance [2]. These contributions support demonstration and debugging without claiming exact workpiece tracking, physical truth or a Digital Twin. If the 2D hierarchy and real-observation refinement are completed with sufficient time remaining, selected modules may be explored in 3D.

8. Conclusion

This Milestone 1 report documents an implemented hierarchical read-only visualisation rather than only a planned status display. The current IP provides a Level-1 production-line overview, clickable Level-2 module dialogs, continuous idealised mechanism animation, one shared observer-side bottle/process model and explicit separation between real Coordinator-provided observations and idealised visual state. The architecture remains one way and sends no command or actuator request back to Coordinator or Controllers. Remaining work is limited to genuine gaps: an approved P6 Labeller monitoring interface and visual module, richer team-agreed telemetry, alignment of the simplified five-station Rotary graphic with the six-position/P6 architecture, and optional downstream Sort/Pack representation. Selected 3D exploration is optional only if time remains; exact workpiece tracking, Digital Twin behaviour and complete real-controller integration are not claimed.

9. References

[1] A. Malik, Z. Salcic, P. S. Roop, and A. Girault, "SystemJ: A GALS language for system level design," *Computer Languages, Systems & Structures*, vol. 36, no. 4, pp. 317-344, Dec. 2010, doi: 10.1016/j.cl.2010.01.001.

[2] International Society of Automation, *ANSI/ISA-101.01-2015: Human Machine Interfaces for Process Automation Systems*. Research Triangle Park, NC, USA: ISA, 2015.

[3] D. V. C. Reising and P. T. Bullemer, "A direct perception, span-of-control overview display to support a process control operator's situation awareness: A practice-oriented design process," *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 52, no. 4, pp. 267-271, Sep. 2008, doi: 10.1177/154193120805200415.

[4] E. Negri, L. Fumagalli, and M. Macchi, "A review of the roles of Digital Twin in CPS-based production systems," *Procedia Manufacturing*, vol. 11, pp. 939-948, 2017, doi: 10.1016/j.promfg.2017.07.198.

[5] C. Cimino, E. Negri, and L. Fumagalli, "Review of digital twin applications in manufacturing," *Computers in Industry*, vol. 113, Art. no. 103130, Dec. 2019, doi: 10.1016/j.compind.2019.103130.